

03. Arquitectura

PDF Reader v0.3.3 | Análisis 2026-09-29 | Referencia del release v0.3.3

Contenido

- Estilo y principios
- Componentes y dependencias
- Secuencia de apertura
- Estado, asincronía y errores
- Autenticación, autorización y confianza
- Despliegue
- Decisiones y consecuencias

Estilo y principios

La arquitectura es un núcleo Web compartido con adaptadores nativos delgados. No hay backend ni base de datos remota. El diseño favorece pocas capas auditables: presentación/orquestación, motor PDF, persistencia local y puentes de plataforma. El ADR docs/adr/0001-cross-platform-shells.md registra esta decisión.

Diagrama Mermaid

```
mindmap
  root((PDF Reader))
    Núcleo Web
      UI HTML/CSS
      app.js
      PDF.js
      IndexedDB
    Android
      Capacitor
      PdfIntentPlugin
      Share
    Windows
      Electron main
      preload IPC
      electron-builder
    Entrega
      CI
      Pages
      Release
```

Componentes y dependencias

Diagrama Mermaid

```
flowchart TB
  HTML["HTML[src/index.html + styles.css]"] --> APP["APP[src/app.js]"]
  APP --> UTIL["UTIL[src/utls.js]"]
  APP --> STORE["STORE[src/history-store.js]"]
  APP --> PDFJS["PDFJS[pdfjs-dist]"]
  PRE["PRE[desktop/preload.cjs]"] --> APP
  MAIN["MAIN[desktop/main.cjs]"] --> PRE
  JAVA["JAVA[PdfIntentPlugin.java]"] --> CAP["CAP[Capacitor bridge]"]
  CAP --> APP
  STORE --> IDB["IDB[(IndexedDB)]"]
  APP --> LS["LS[(localStorage)]"]
```

app.js mantiene un único estado de sesión y enlaza eventos DOM. PDF.js se carga como módulo/worker local. history-store.js encapsula transacciones IndexedDB. Electron expone solo cinco operaciones por contextBridge; Android registra un plugin que lee la URI autorizada y devuelve Base64.

Secuencia de apertura

Diagrama Mermaid

```
sequenceDiagram
    actor U as Persona
    participant A as Adaptador
    participant C as app.js
    participant P as PDF.js
    participant D as IndexedDB
    U->>A: elige/abre PDF
    A->>C: bytes + metadatos
    C->>P: getDocument(data)
    P-->>C: PDFDocumentProxy
    C->>D: guarda copia y progreso
    C->>P: getPage + render
    P-->>U: canvas visible
```

Estado, asincronía y errores

El objeto `state` concentra documento, archivo, página, zoom, ajuste, rotación, tarea de render, caché de texto y gestos. Las operaciones PDF/IndexedDB son promesas. `renderSeq` y la cancelación de `renderTask` evitan que un render anterior sobrescriba al más reciente. Los errores de contraseña/formato se traducen a mensajes de usuario; un fallo de persistencia no impide leer.

No hay colas, workers propios ni procesos de fondo aparte del worker de PDF.js y los jobs de CI. Miniaturas y búsqueda ceden control al navegador periódicamente con `requestAnimationFrame`.

Autenticación, autorización y confianza

No existen usuarios, sesiones ni roles. La autorización de archivo la concede el selector del sistema o una asociación explícita. Electron aísla el renderer (`contextIsolation`, sin Node, sandbox). Android no declara Internet ni almacenamiento general. El contenido PDF se considera no confiable y queda dentro de PDF.js/renderer.

Despliegue

Diagrama Mermaid

```
graph LR
    G[Git main/tag] --> CI[GitHub Actions]
    CI --> P[GitHub Pages: landing + app]
    CI --> W[Windows NSIS + portable]
    CI --> A[APK Android firmado]
    W --> R[GitHub Release]
    A --> R
    R --> H[SHA256SUMS.txt]
```

`ci.yml` valida Web, Windows y Android; `pages.yml` despliega desde `main`; `release.yml` produce binarios al etiquetar. No se identificaron contenedores, infraestructura cloud propia, caché remota ni observabilidad de runtime.

Decisiones y consecuencias

- Vanilla JS reduce dependencias y jerarquía, pero `app.js` concentra demasiadas responsabilidades.
- Compartir el núcleo garantiza paridad, pero hereda límites de WebView y memoria.
- Persistir bytes facilita reanudar sin permisos amplios, pero consume cuota IndexedDB.
- No tener backend minimiza exposición y operación, a costa de no sincronizar dispositivos.